



WAYPOINT CONNECT

PROJECT DOCUMENTATION / README

Project Documentation

Overview, features, workflow, status, stack, setup and accounts

Project WaypointConnect — delivery planning and tracking for Waypoint Group's Fresh, Style and Tech brands

Event Tech-Triathlon 2026 · Team Dark Horse

Source Generated 2026-10-04 · repository commit c1e3eb6 + working tree

Accuracy rule: every statement is taken from the project files. Anything that could not be verified is marked "Not verified from the provided project files". No passwords or secrets are included.

Contents

1. Project overview	2
1.1 Project at a glance	2
1.2 Problem statement	2
1.3 Proposed solution	3
1.4 Main objectives	3
1.5 Target users and user roles	3
2. Features	4
2.1 Main system features	4
3. Business workflow	6
3.1 Business workflow	6
4. Business rules	7
4.1 Important business rules (verified in source)	7
5. Implementation status	9
5.1 Current implementation status	9
6. Technology stack and requirements	10
6.1 Technology stack	10
6.2 System requirements	10
7. Local setup and environment	11
7.1 Prerequisites	11
7.2 Environment configuration	11
7.3 Repository, install, Prisma, database, seed	12
7.4 Running the API and the web app	12
7.5 Tests and checks	12
7.6 Production build	12
8. Docker and deployment	13
8.1 Running with Docker Compose	13
8.2 Deployment	13
9. Accounts and walkthrough logins	14
9.1 Test accounts and walkthrough accounts	14

1. Project overview

1.1 Project at a glance

Name	Waypoint Connect (repository <code>DarkHorse_WaypointConnect</code> , workspace package <code>waypoint-connect</code> v0.1.0)
Event / team	Tech-Triathlon 2026, Team Dark Horse (<code>README.md</code> , root <code>package.json</code>); <code>docs/handoff.md</code> calls it the Hackathon part of Rootcode Tech-Triathlon 2026
Purpose	Delivery planning and tracking for Waypoint Group's three brands (Fresh, Style, Tech): one system for the dispatcher, the loader, the driver (works offline) and the store manager. An administrator role was added later.
Scale in the data	120 outlets (<code>data/outlets.csv</code>), 60 vehicles (<code>data/vehicles.csv</code>), 2 depots (Peliyagoda, Kandy), 188 sign-in accounts (README)
Live demo	https://3-106-77-179.sslip.io (from <code>README.md</code> ; it answered HTTP 200 when the documentation was generated on 2026-10-04). Driver phone app page: <code>/get-app</code>
Repository	https://github.com/Cognix-0/DarkHorse_WaypointConnect (from <code>docs/handoff.md</code> ; whether it is public <i>Not verified from the provided project files.</i>)

1.2 Problem statement

Waypoint Group delivers goods for three brands from two depots. Every day each order has to be assigned to a vehicle and a trip without breaking operating rules that the repository records as coming from the Challenge Booklet (`CLAUDE.md`, `packages/shared/src/engine/rules.ts`):

- weight and volume limits per vehicle, chilled goods only on refrigerated (reefer) vehicles, shops that can only be reached by van, and each vehicle serving only its home depot;
- at most two trips per vehicle per day, one brand and one district per trip, and orders are never split;
- time limits: the Fresh window 03:30–08:00 (270 minutes) and a 480-minute day for Style and Tech, plus a weekly fuel quota per vehicle;
- an order cutoff at 16:00 Sri Lanka time.

On a peak day the chilled demand exceeds the refrigerated capacity: the README describes the official peak-day scenario as about 32.8 t of chilled demand against 3 refrigerated trucks and 1 refrigerated van, so some orders cannot legally be served. Those deferrals need a reason, and shops that were skipped before must not be silently skipped again. Four groups of people must see the same facts at the same time, and drivers must be able to keep working when the phone has no signal.

Source note

The original Challenge Booklet and Designathon proposal were **not** among the provided files. This problem statement is reconstructed from `README.md`, `CLAUDE.md`, `docs/handoff.md` and the code. Statements about the booklet are marked as "recorded in the repository".

1.3 Proposed solution

Waypoint Connect is one web application (a React single-page app that is also an installable PWA) with a Fastify API and a PostgreSQL database. A planning engine in `packages/shared/src/engine` contains every planning rule and is used by both the API and the browser, so the dispatcher, the loader, the driver and the store manager always see the result of the same rules. The system follows the order through four steps that are also shown on the sign-in page: **Order** → **Plan** → **Load** → **Deliver**, followed by the store's receipt confirmation.

1.4 Main objectives

The README and code do not contain a formal objectives list. The objectives below are derived from what the system implements:

1. Let each store order from a goods catalogue (counted in packs) until the 16:00 cutoff, and follow the order to delivery.
2. Let the dispatcher see every order in priority order, auto-allocate a plan, change it by drag and drop with every drop checked by the engine, and publish it with a stated reason for every deferral.
3. Let loaders load by goods line, report problems, and seal and release a trip, with plan changes shown as a to-do list.
4. Let drivers deliver with proof of delivery, including with no signal, without losing or double-sending any event.
5. Let store managers confirm what arrived line by line and answer a deferral.
6. Let an administrator manage accounts, put one driver on each vehicle, check system health and run the demo.

1.5 Target users and user roles

Role	Accounts (README)	Works on	Web area
Administrator	<code>admin@waypoint.lk</code> (1; more can be created)	Accounts, drivers on vehicles, system health, demo mode	<code>/admin</code>
Dispatcher	<code>dispatcher@waypoint.lk</code> (1)	Both depots: switches between Peliyagoda and Kandy in the console; plans, publishes, tracks	<code>/dispatcher</code>
Loader	<code>loader1.peliyagoda@waypoint.lk</code> ... <code>loader3.kandy@waypoint.lk</code> (3 per depot)	The loader's own depot: loads and seals trips	<code>/loader</code>
Driver	<code>veh001@waypoint.lk</code> ... <code>veh060@waypoint.lk</code> (1 per vehicle)	Signs in as the vehicle; shows the vehicle's assigned driver; delivers (offline capable)	<code>/driver</code>
Store manager	<code>out001@waypoint.lk</code> ... <code>out120@waypoint.lk</code> (1 per store)	Own store only: orders, tracking, receipts	<code>/store</code>

2. Features

2.1 Main system features

Dispatcher console (screens D1–D6)

Screen	What it does (README)
D1 Overview	Today's numbers, demand by brand, both depots, what needs attention
D2 Order Queue	Locked orders in priority order with filters and CSV export; orders placed after the 16:00 cutoff greyed out; Auto-allocate & open board
D3 Planning Board	Drag stops between vehicles and trips; every drop is checked by the engine and a refused drop shows Drop blocked with the exact rule; weight, volume and time meters; weekly fuel per vehicle; ETAs per stop
D4 Deferrals	Overload banner, three recovery options re-planned by the engine, the deferral table, explicit confirmation for a shop skipped twice, Confirm & publish
D5 Live Tracking	Trip progress, ETA against each shop's window (on time / at risk / late / offline), alerts with actions, View POD
D6 Capacity Forecast	Next 3 weeks: chilled demand against reefer space and projected deferrals from the same engine

Loader dock tablet (L1–L5)

Choose vehicle (cards with bay, departure and progress) → Load goods (lines in packs grouped by stop, last stop first, tick each line) → Report a problem (damaged, short, wrong pack..., with action carry over / replace / hold, photo and note) → Plan changes (plan v1 → v2 to-do list, mark moved, acknowledge) → Seal & release (seal number, reefer temperature for reefer vehicles; short packs become an order on the next run).

Driver phone app (M1–M8)

Today's runs → Load check → Route progress (Run / Stops / Sync tabs) → Navigate (opens Google Maps) → Stop arrival → Proof of delivery (delivered / partial / refused, count, camera photo shrunk to about 100 kB, receiver, signature) → Report a problem → Offline sync queue → Back online sheet. Every action is saved on the phone first and synchronised later (section on offline sync in the architecture document).

Store manager (SM1–SM9, phone and desktop)

Today (cutoff countdown, today's deliveries, tomorrow, this week, activity) → Place order from the goods catalogue (dry and chilled ordered separately; copy last order; live weight and volume; editable until 16:00) → Order status and tracking (Received → Scheduled → Loaded → Delivered) → Confirm receipt line by line (OK / Short / Damaged) → Order deferred (accept the new day or cancel) → Orders, Receipts and Help.

Administrator console

System (database and API health, working day and clock, today's orders and plan per depot, *Build today's orders*), **Vehicles & drivers** (one driver per vehicle, move with confirmation, add drivers), **Demo mode** (start, extend, reset, end) and **Accounts** (search, switch on/off, reset password shown once, create dispatcher / loader / administrator accounts).

Cross-cutting features

- **Real-time refresh:** every screen refreshes without a reload through a Server-Sent Events signal ([GET /api/changes](#)); the signal carries no business data.
- **Offline-first driver app:** service worker ([apps/web/public/sw.js](#)), IndexedDB outbox ([src/offline/idb.ts](#)) and an idempotent [POST /api/sync](#) .
- **Plan versions:** every publish stores a snapshot; loaders see the difference and acknowledge it; sealed or departed trips are locked against changes.
- **Fingerprint / Face ID app lock** for the driver app (WebAuthn, local check only; the password and 12-hour token remain the real login).
- **Get the app page** ([/get-app](#)): Android APK download or Add to Home Screen on iPhone.
- **Sri Lanka time everywhere** (Asia/Colombo) regardless of the device time zone.

3. Business workflow

3.1 Business workflow

1. **Order.** A store manager places a dry (and, for Fresh stores, a chilled) order from the goods catalogue. Before 16:00 it is for the next operating day; after 16:00 it goes to the following run.
2. **Lock.** Orders placed before the cutoff are the run's locked orders; later ones are shown greyed out in the queue.
3. **Plan.** The dispatcher runs Auto-allocate, adjusts the board by drag and drop (each drop checked by the engine), reviews deferrals and presses Confirm & publish. Every affected store is told; each deferred order becomes a carry-over order on the next run.
4. **Load.** Loaders claim a vehicle trip, tick each goods line (or report a problem), and seal and release the trip.
5. **Deliver.** The driver confirms the load, departs, and records arrival, proof of delivery or a problem at each stop, with or without signal.
6. **Confirm.** The store manager confirms receipt line by line; any short or damaged line raises a Store issue alert for the dispatcher.

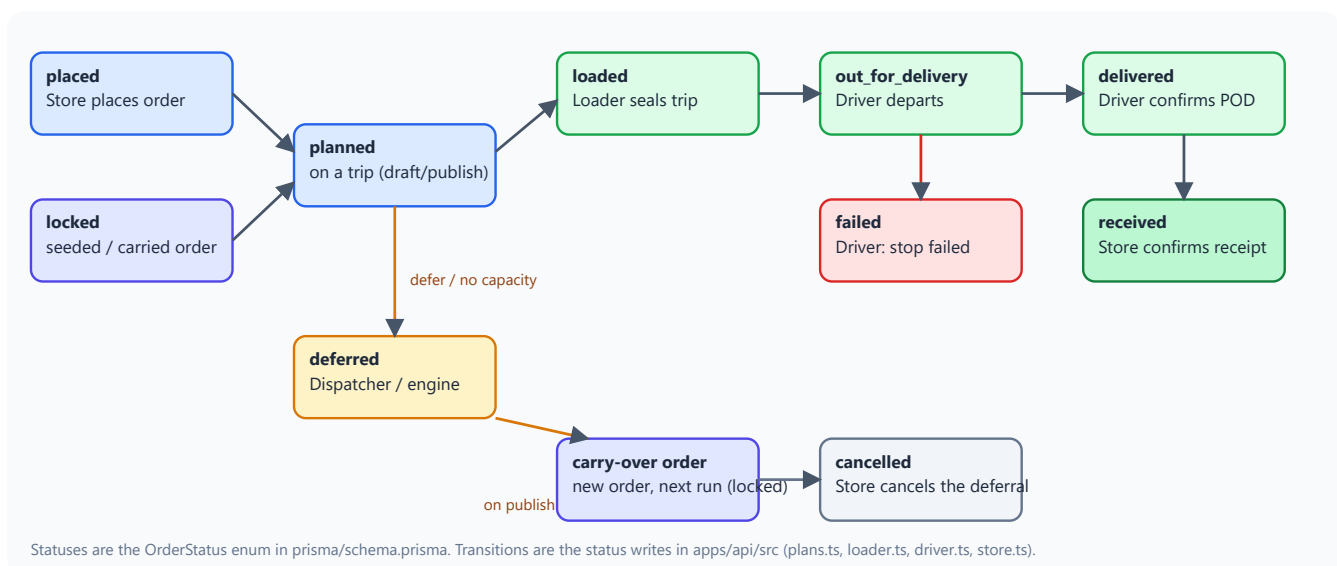
Source: apps/api/src/reference.ts (workingDay, ORDER_CUTOFF, nextOperatingDay) and apps/api/src/routes/store.ts (ctxOf)

"Next operating day" skips non-operating days in data/calendar.csv.



At midnight the calendar day rolls over and the day is built automatically on the first request (builtDays in server.ts).

The 16:00 cut-off and the working run of each role.



Order status lifecycle (OrderStatus enum and the status writes in the API).

4. Business rules

4.1 Important business rules (verified in source)

Rule	Behaviour	Where
Time zone	All business logic and every screen use Asia/Colombo (UTC+5:30). The database stores UTC; the web screens format times with <code>colomboTime</code> / <code>colomboHour</code> .	<code>apps/api/src/reference.ts</code> , <code>apps/web/src/ui/format.ts</code> , <code>docker-compose.yml</code>
Order cutoff	The cutoff is 16:00 on the day before delivery (10:30 UTC).	<code>cutoffFor</code> , <code>ORDER_CUTOFF</code> in <code>reference.ts</code>
Current run (dispatcher, loader, driver)	Today's date before 16:00; from 16:00 the next operating day , whose orders are now locked, so the dispatcher plans and publishes it in the evening.	<code>workingDay()</code> in <code>reference.ts</code>
Store manager day	The store works on the calendar day. Before 16:00 an order is for the next operating day; from 16:00 it goes on the run after that. It can be changed until the cutoff.	<code>ctxOf()</code> in <code>routes/store.ts</code>
Next operating day	Skips days that are not operating days in <code>data/calendar.csv</code> (Sundays and holidays).	<code>nextOperatingDay()</code>
Next-run queue	A store order placed after the cutoff appears at once on the dispatcher's Order Queue and Overview under "next run" (greyed out) until it locks, then in the queue to plan.	README; <code>loadDay()</code> in <code>day.ts</code> (<code>afterCutoff</code>)
Daily order build	Each day is filled with the 85 Peliyagoda orders and workshop vehicles of the official peak-day scenario (S1), built automatically on the first request of a new day and at the run switch. It never adds duplicates.	<code>prisma/demo-day.ts</code> , <code>server.ts</code> (<code>ensureToday</code>)
DEMO_DATE	If set to YYYY-MM-DD, every screen runs on that fixed day instead of today (also <code>DEMO_CLOCK</code> , <code>DEMO_LOADER_CLOCK</code> , <code>DEMO_STORE_CLOCK</code> pin a role's clock). Empty = real date and time.	<code>demoDate()</code> , <code>reference.ts</code> , <code>.env.example</code>
Demo mode (admin)	Runs the whole system on the peak day Wed 25 Mar 2026 for 0.25–12 hours; store at 14:18 on the day before, loader 02:40, dispatcher 06:42. Ends by itself or with End demo now ; Reset demo data returns to the starting point.	<code>apps/api/src/demo.ts</code> , <code>StartDemoRequest</code> in <code>contract.ts</code>
Priority order	1 skipped on the last run (or Fresh shops unserved 2+ days); 2 Fresh chilled; 3 Fresh ambient; 4 Tech; 5 Style. Tie-breaks: van-only, then mall slot, earliest window close, heaviest first.	<code>engine/priority.ts</code>
Vehicle and trip rules	Chilled needs a reefer; van-only outlets need a van; a vehicle serves only its home depot; workshop vehicles are refused; max 2 trips per vehicle per day; one brand and one district per trip; weight and volume caps; weekly fuel quota; orders are never split (one stop per order).	<code>engine/rules.ts</code> (codes such as <code>NEEDS_REEFER</code> , <code>MAX_TRIPS</code>)
Time limits	Trip minutes = depot-to-district free-flow + inter-stop free-flow × (stops – 1) + service allowance per stop. Fresh trips must fit 270 minutes (03:30–08:00); Style and Tech 480 minutes.	<code>engine/rules.ts</code> , <code>engine/schedule.ts</code> ; tests

Rule	Behaviour	Where
Deferrals	Every deferral has a reason and a type (<code>unavoidable</code> or <code>choice</code>). A shop skipped twice needs an explicit dispatcher confirmation before publishing. Publishing carries each deferred order to the next run and tells the store.	<code>schema.prisma</code> (<code>Deferral</code>), <code>plans.ts</code>
Locked trips	A sealed or departed trip is locked: the board, defer and auto-allocate refuse to change it.	<code>routes/dispatch.ts</code> (<code>closedTrips</code>), CLAUDE.md
Seal rules	A trip can only be sealed when every line is ticked or reported; reefer vehicles need a logged temperature. Short packs become a new order on the next run.	<code>routes/loader.ts</code> (<code>seal</code>)
Goods in packs	Packs on an order's lines always add up to its units (tested).	<code>packages/shared/src/goods.ts</code>
Admin driver rule	One driver per vehicle; a driver cannot be on two vehicles (database unique index on <code>Vehicle.driverId</code>); moving a driver needs confirmation.	<code>schema</code> , <code>routes/admin.ts</code>
Account switch-off	A switched-off account cannot sign in and an open session stops at its next request.	<code>auth.ts</code> (<code>requireRole</code>)

5. Implementation status

5.1 Current implementation status

Area	Status	Evidence
Monorepo, contract, schema, seed, planning engine, login API, web shell (Part 1)	IMPLEMENTED	README status checklist; 21 engine tests pass (run on 2026-10-04)
Dispatcher API and screens D1–D6 (Part 2)	IMPLEMENTED	README checklist; <code>apps/api/src/routes/dispatch.ts</code> , <code>insights.ts</code> , <code>live.ts</code>
Loader L1–L5 and offline driver app M1–M8 (Part 3)	IMPLEMENTED	README checklist; <code>routes/loader.ts</code> , <code>routes/driver.ts</code> , <code>roles/loader</code> , <code>roles/driver</code>
Store manager SM1–SM9 and API, judge walkthrough, deployment guide, video script (Part 4)	IMPLEMENTED	README checklist; <code>routes/store.ts</code> , <code>docs/deploy.md</code> , <code>docs/video-script.md</code>
Administrator console, per-person accounts, Demo mode, real-time refresh	ADDED LATER	<code>docs/ai-disclosure.md</code> entries of 2026-10-04; <code>routes/admin.ts</code>
Run on a real server	IMPLEMENTED	ai-disclosure entry "deployed to AWS Lightsail"; the URL answered HTTP 200 on 2026-10-04. The README checklist item is still unticked.
Record the video and submit	NOT VERIFIED	README checklist item is unticked; a script exists (<code>docs/video-script.md</code>), a recorded video is not among the files

Known documentation drift in the repository

- `README.md` still describes a **Simulate morning** button in Live Tracking. The latest commits remove it from the screen (commit 3b1c40f); `useSimulate` in `roles/dispatcher/api.ts` is no longer used, while `POST /api/live/simulate` remains in the API.
- `README.md` quotes both "72 of 85 orders on 32 trips" (engine on the official peak day, reproduced on 2026-10-04) and "75 of 85 orders on 34 trips" (in the app, explained by the README as a difference in the skipped-shop flag).
- `docs/handoff.md` still lists four shared logins (`@waypoint.demo`) and a shared `DEMO_PASSWORD` ; they were replaced by personal accounts on 2026-10-04.
- `docs/deploy.md` describes DigitalOcean; the AI disclosure records the live deployment on AWS Lightsail.
- The untracked folder `prisma/migrations/20261004071614_init` creates 17 tables and does not contain `Driver` or `Setting` ; the Docker setup uses `prisma db push` , not migrations.

6. Technology stack and requirements

6.1 Technology stack

Layer	Technology	Version source
Language	TypeScript, strict, everywhere (pnpm workspaces)	root <code>package.json</code> : typescript ^5.6.3
Frontend	React 18, Vite 6, Tailwind CSS 3, React Router 6, TanStack Query 5	<code>apps/web/package.json</code>
Offline / PWA	Service worker <code>public/sw.js</code> , IndexedDB outbox (no dependency), web manifest	<code>apps/web</code>
Backend	Fastify 5, @fastify/jwt 9, @fastify/cors 10, Zod 3, bcryptjs 2, tsx	<code>apps/api/package.json</code>
ORM / database	Prisma 6 and PostgreSQL 16	<code>prisma/schema.prisma</code> , <code>docker-compose.yml</code> (<code>postgres:16</code>)
Shared library	Zod API contract + dependency-free planning engine	<code>packages/shared</code>
Container runtime	Docker Compose: db, api (<code>node:20-slim</code>), web (<code>nginx:1.27-alpine</code>), caddy (<code>caddy:2</code>)	<code>docker-compose.yml</code> , Dockerfiles
Tests	Node built-in test runner (<code>node --import tsx --test</code>)	<code>packages/shared/package.json</code>
CI	GitHub Actions: tests, Prisma generate, typecheck, official allocation checker, Docker build	<code>.github/workflows/ci.yml</code>
Android app	Trusted Web Activity built with Bubblewrap in GitHub Actions (package id <code>lk.darkhorse.waypointconnect</code>)	<code>docs/handoff.md</code> , <code>android-app.yml</code>

6.2 System requirements

Requirement	Value	Source
Node.js	20 or newer	root <code>package.json</code> <code>engines</code> ; Dockerfiles use Node 20. The verification run for this document used Node v24.21.0.
Package manager	pnpm 9.12.0 (enable with <code>corepack enable</code>)	<code>packageManager</code> ; README
Docker	Docker with Compose v2 (<code>docker compose</code>); Docker 29.8.1 was present on the verification machine	README, <code>docs/deploy.md</code>
PostgreSQL	16 (from Docker), or any local PostgreSQL with <code>DATABASE_URL</code> set	README "Develop"
Python 3 + pandas	Only for <code>pnpm check:2b</code> (official checker)	<code>.github/workflows/ci.yml</code>
Browser	Current Chrome or Safari; HTTPS is required for the offline service worker and the camera	<code>docs/deploy.md</code> , <code>Caddyfile</code>
Server (public deployment)	2 vCPU, 2 GB RAM, 20 GB disk, ports 22, 80, 443	<code>docs/deploy.md</code>

7. Local setup and environment

7.1 Prerequisites

Item	Requirement	Check
Node.js	20 or newer (<code>engines</code>); Docker images use Node 20	<code>node -v</code>
pnpm	9.12.0 via Corepack (<code>packageManager</code>)	<code>corepack enable</code> then <code>pnpm -v</code>
Docker	Docker with Compose v2	<code>docker compose version</code>
PostgreSQL	16 (Docker <code>db</code> service) or a local server	see Database setup
Python 3 + pandas	Only for <code>pnpm check:2b</code>	
Git	To clone the repository	

7.2 Environment configuration

Copy the example and change every `CHANGE_ME` value. Docker Compose reads `.env` in the project folder; `pnpm dev:api` reads it through `--env-file=../../.env`. `.env` is gitignored; never commit it.

```
cp .env.example .env          # Windows PowerShell: Copy-Item .env.example .env
```

Variable	Meaning	Default / note
<code>DOMAIN</code>	Public domain for Caddy (no https://)	<code>localhost</code>
<code>DB_PASSWORD</code>	PostgreSQL password	compose default <code>waypoint</code> . Must equal the password inside <code>DATABASE_URL</code>
<code>DATABASE_URL</code>	Connection string for local (non-Docker) API / Prisma	<code>postgresql://postgres:CHANGE_ME@localhost:15432/waypoint</code>
<code>JWT_SECRET</code>	Signs the 12-hour session tokens	compose fallback <code>dev-only-secret-change-me</code> : always change on a server
<code>ACCOUNT_SECRET</code>	Derives every account password	falls back to <code>JWT_SECRET</code> ; changing it changes all passwords
<code>API_PORT</code>	API port	3000
<code>DEMO_DATE</code>	Pin all screens to one day (YYYY-MM-DD)	empty = real date
<code>DEMO_MODE</code>	Enables the demo-only simulate route	compose default 1
<code>DEMO_CLOCK</code> , <code>DEMO_LOADER_CLOCK</code> , <code>DEMO_STORE_CLOCK</code>	Pin a role's clock (HH:MM)	empty = real Colombo time
<code>DB_HOST_PORT</code>	Host port for the database	15432
<code>LOG_LEVEL</code>	API log level (read by the API)	info

7.3 Repository, install, Prisma, database, seed

```
git clone https://github.com/Cognix-0/DarkHorse_WaypointConnect.git
cd DarkHorse_WaypointConnect
corepack enable
pnpm install
cp .env.example .env          # then edit the CHANGE_ME values
docker compose up -d db       # PostgreSQL on 127.0.0.1:15432 (or use any local PostgreSQL and
set DATABASE_URL)
pnpm prisma generate
pnpm prisma db push           # creates the tables from prisma/schema.prisma
pnpm db:seed                  # outlets, vehicles, drivers, 188 accounts, today's orders
```

The scripts are from the root `package.json`: `db:seed = tsx prisma/seed.ts`; `credentials = tsx scripts/credentials.ts`. The seed is safe to run repeatedly. It needs `ACCOUNT_SECRET` or `JWT_SECRET` in the environment (otherwise it stops with "Set ACCOUNT_SECRET..."). A `db:migrate` script (`prisma migrate dev`) exists, but Docker and the README use `db push`.

7.4 Running the API and the web app

```
pnpm dev:api    # http://localhost:3000/api/health
pnpm dev:web    # http://localhost:5173 (proxies /api to localhost:3000)
```

Print the account passwords with `pnpm credentials` (output is CSV: role, depot, vehicle_or_store, name, email, password). Keep it private.

7.5 Tests and checks

```
pnpm test      # 21 planning-engine tests
pnpm typecheck # all three workspaces (run pnpm prisma generate first)
pnpm check:2b  # engine on the official peak day + check_allocation.py (needs Python 3 + pandas)
```

7.6 Production build

```
pnpm --filter @waypoint/web build # tsc + vite build into apps/web/dist
```

The API is run from TypeScript with `tsx` (`pnpm --filter @waypoint/api start`); there is no separate API build step.

8. Docker and deployment

8.1 Running with Docker Compose

```
cp .env.example .env
docker compose up --build      # database + API (schema + seed) + web + HTTPS
docker compose up -d --build   # detached; rebuild after code changes
docker compose logs --tail=50 api # wait for the seed and "Server listening"
docker compose down -v && docker compose up -d --build # wipe the database and re-seed
```

Open <https://localhost> and accept the local certificate once. Only Caddy publishes ports 80 and 443. After `git pull`, rebuild with `docker compose up -d --build`; on phones close and reopen the app once.

8.2 Deployment

From [docs/deploy.md](#). Server: Ubuntu 24.04, 2 vCPU, 2 GB RAM, 20 GB disk, ports 22/80/443 open.

One command

```
curl -fsSL https://raw.githubusercontent.com/Cognix-0/DarkHorse_WaypointConnect/main/scripts/server-setup.sh | bash
# with a domain: curl ... | DOMAIN=waypoint.example.com bash
```

It prints `Live: https://...` after 5–10 minutes; without a domain it uses `<ip-with-dashes>.sslip.io`. Run it again to deploy a new commit.

By hand

1. Point an A record at the server (`dig +short your.domain` must print the IP).
2. Install Docker: `curl -fsSL https://get.docker.com | sudo sh`.
3. Clone, `cp .env.example .env`, set `DOMAIN`, `DB_PASSWORD`, `JWT_SECRET`, `ACCOUNT_SECRET` (e.g. `openssl rand -hex 32`).
4. `docker compose up -d --build` then `docker compose logs -f api`.
5. List passwords: `docker compose exec api npm -s credentials > credentials.csv` (keep private).

Checklist before submitting: padlock on the HTTPS URL; walkthrough accounts sign in; Order Queue shows 85 orders; driver app loads offline on a phone. Live deployment recorded in this project: <https://3-106-77-179.sslip.io>.

9. Accounts and walkthrough logins

9.1 Test accounts and walkthrough accounts

Every account has its own password. Passwords are **never stored in plain text**: each is derived from `ACCOUNT_SECRET` in `.env` and the account email (`prisma/accounts.ts`). They are printed with `pnpm credentials` ; on a server: `docker compose exec api pnpm -s credentials > credentials.csv` . The README states that judges receive the walkthrough passwords with the submission. No password is reproduced in this document.

Role	Walkthrough account	Linked to
Administrator	<code>admin@waypoint.lk</code>	everything
Dispatcher	<code>dispatcher@waypoint.lk</code>	both depots (depot switcher in the console)
Loader	<code>loader1.peliyagoda@waypoint.lk</code>	Peliyagoda depot
Driver (vehicle)	<code>veh024@waypoint.lk</code>	VEH024, dry truck, driver "Nimal Fernando"
Store manager	<code>out026@waypoint.lk</code>	OUT026 Waypoint Fresh, Gampaha; "Nimali Perera"

Authentication behaviour found in the source (see the Testing report)

In `apps/api/src/routes/auth.ts` the sign-in check treats **any non-empty password as valid** for an existing account, and unknown `dispatcher...` , `loader...` , `veh...` , `out...` , `admin...` emails fall back to the first account of that role. This contradicts the README statement that passwords are unique per account. The finding is documented as observed; it should be fixed before the system is treated as secure.